BIM-Native Tokenization for Constraint-Aware Room Layout Synthesis

Manuel Ladron de Guevara*, Jinmo Rhee†, Ardavan Bidgoli*, Vaidas Razgaitis*, Michael Bergin*
*Higharc, United States †University of Calgary, Canada
{manuelrodriguez, ardavanbidgoli, vaidasrazgaitis, michaelbergin}@higharc.com jinmo.rhee@ucalgary.ca
Project page: https://manuelladron.github.io/sbm

***Abstract*—We present a BIM-native tokenization for room-level layout synthesis in Building Information Modeling (BIM) scenes. The core contribution is representational: we encode each room as a sequence of *BIM-Token Bundles*, realized as columns of a sparse attribute–feature matrix that unifies categorical and continuous attributes of walls, openings, and entities under wall-referenced (translation/scale-invariant) coordinates. A mixed-type embedding module produces a unified token vector from this matrix; a single Transformer backbone is then trained in two modes: encoder-only for room embeddings and retrieval, and encoder–decoder for autoregressive entity placement, which we call Data-Driven Entity Prediction (DDEP). On a controlled same-data benchmark with shared ontology and evaluation harness, DDEP outperforms ATISS and BLT baselines bridged into our representation, with ablations identifying joint continuous-feature embedding and entity ordering as primary drivers. Encoder embeddings cluster rooms by type more tightly than large general-purpose text encoders, which in turn retain an edge on within-type ranking. We frame this work as evidence that modestly sized, domain-specific sequence models over well-designed BIM tokenizations are a useful primitive for constraint-aware spatial generation, complementary to general-purpose LLMs/VLMs which we also benchmark.**

***Index Terms*—BIM, layout synthesis, structured tokenization, Transformer, constraint-aware generation.**

I. Introduction

Computer-Aided Design (CAD) and Building Information Modeling (BIM) scenes encode rich semantics, hierarchies, and domain constraints that go well beyond appearance. Most strong 3D generators—voxel, mesh, point-cloud, and image-conditioned diffusion models—treat scenes as unstructured geometry [2], yielding visually plausible but hard-to-edit outputs that often violate basic validity rules. Room-level layout design, in contrast, requires reasoning over semantics, references, and constraints at multiple scales, and producing *parametric* outputs that downstream BIM tools can ingest and modify. Decades of computational approaches—from CAD macros and parametric families to rule systems, optimization, and learning-based methods—have aimed to automate this process [3]–[5], yet progress remains bottlenecked less by the algorithm and more by the representation that the algorithm operates on. An effective representation must expose structure, generalize across typologies, and remain BIM-editable.

We argue that for room-level BIM layout this representation should be *sequence-native*, *mixed-type*, and *wall-referenced*. Sequence-native, so a single Transformer backbone can serve retrieval and generation; mixed-type, so categorical (room type, family identifiers) and continuous (positions, sizes, rotations) attributes share one embedding pipeline; and wall-referenced, so spatial coordinates are invariant to absolute translation and scale and align with architectural reasoning. We instantiate these properties in a normalized tokenizer that emits *BIM-Token Bundles*—sequence units encoding room topology, entity attributes, wall-referenced geometry, and relational structure—realized as columns of a sparse attribute–feature matrix, whose rows are token features, and fused via a mixed-type embedding module into a unified token vector. Unlike flat serializations, this matrix keeps every entity editable as a BIM object: an output token still carries its category, hosting wall, parametric position, offset, dimensions, and rotation rather than only an image-space box. We then train a single Transformer backbone in two modes: encoder-only for room embeddings and retrieval, and encoder–decoder for autoregressive entity placement, which we call Data-Driven Entity Prediction (DDEP); Fig. 1 shows representative outputs.

We make three contributions. (1) A BIM-native, wall-referenced tokenization of residential room layouts paired with a mixed-type embedding module that jointly handles categorical, scalar, and grouped-continuous features. (2) A single Transformer backbone reused across retrieval and generation, with constrained decoding for BIM validity at inference. (3) A two-track evaluation: a *controlled same-data benchmark* that isolates architectural effects under matched data and budgets, plus a deployment-realistic comparison against frontier LLMs/VLMs and domain-specific layout generators. We frame the LLM/VLM comparison as a calibration baseline rather than a fair architectural comparison, and discuss its limitations explicitly (Section IV). The aim of the paper is not to claim a new general Transformer architecture, but to show that a careful BIM-native tokenization is a useful primitive for constraint-aware indoor layout synthesis.

II. Related Work

a) Rule-based and optimization-based layout.: Early furniture-layout systems encoded interior-design guidelines as cost functions or grammars and solved them with sampling or evolutionary search [4], [6], [7]. These approaches embed human design knowledge and remain interpretable, but are bounded by their predefined rules and require manual rule authoring per typology.

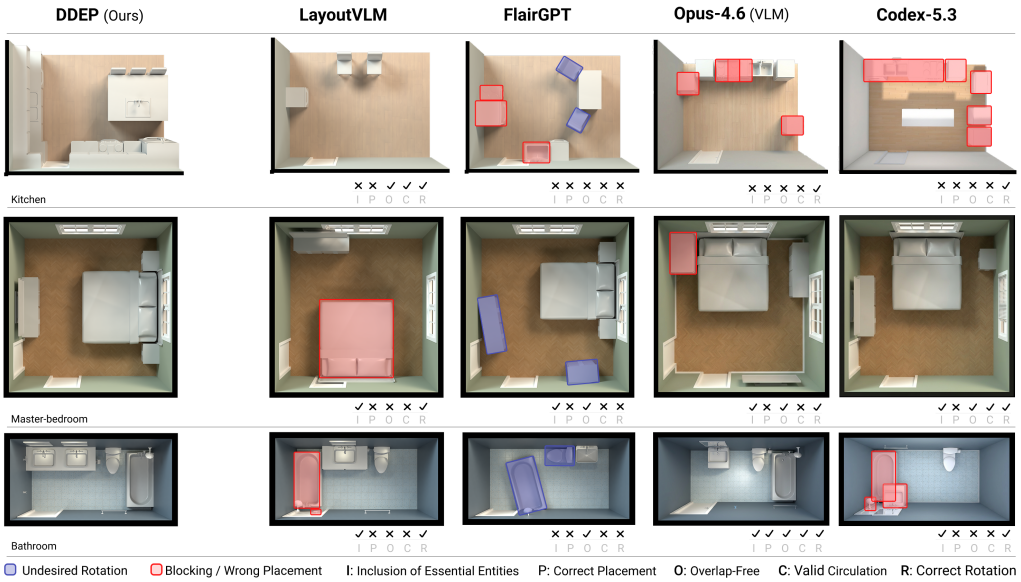

Fig. 1: Qualitative comparison. Each row shows a different room type; columns show outputs from frontier LLM/VLM baselines, domain-specific methods, and our DDEP. DDEP layouts respect wall-referenced anchoring and door-connected circulation while satisfying the inventory program.

b) Deep generative layout models.: Graph-based generators model layouts as scene graphs and predict node attributes and relational edges [8], [9]. Image-based approaches [10], [11] and diffusion models [12]–[18] learn strong priors over floorplans and 3D scenes. We differ from floorplan generators such as HouseDiffusion [13]: our output is a sequence of *wall-referenced parametric entities* hosted on room walls rather than a polygonal floorplan, so direct comparison requires output-format bridging (Section IV).

c) Transformer-based indoor scene synthesis.: ATISS [19] and SceneFormer [20] treat indoor scene synthesis as autoregressive generation of furniture objects with discretized/normalized geometric attributes, conditioned on a room boundary. CLIP-Layout [21] augments this with CLIP-based style embeddings. BLT [22] performs masked, iterative layout prediction for graphic design. These methods assume flat object vocabularies with absolute or grid-aligned poses; they do not preserve BIM's heterogeneous, wall-hosted, mixed categorical–continuous structure for downstream editing. We compare directly against ATISS and BLT under a shared harness, with the bridging caveat discussed in Section IV.

d) LLM/VLM-based layout generation.: LayoutGPT [23] casts layout synthesis as compositional planning over object–relation tokens; LayoutVLM [24] couples vision and language features to optimize 3D scenes; HouseTune [25] and FlairGPT [26] use LLMs for floorplan refinement and stylistic exploration. LLM4CAD [27] tokenizes CAD elements symbolically. FloorPlan-DeepSeek [28] performs autoregressive next-room prediction at the floorplan level. None of these unify semantic, relational, and wall-referenced continuous attributes in a single sequence-native form. Our LLM/VLM benchmark (Section IV) is best read as a deployment-realistic floor; we explicitly do not claim it is a fair architectural comparison, as fully-trained DDEP is matched against off-the-shelf zero-shot models.

III. Method

Figure 2 gives the pipeline. A room is first converted into typed tokens. The *encoder* reads only the room envelope—walls, doors, windows—and compresses it into a contextual memory plus a pooled room embedding used for retrieval. The *decoder* reads the sequence of room contents and, attending to that memory, predicts each next entity with its continuous placement parameters. Both modes share one tokenization and one set of weights, differing only in which sequence they read and which heads they use.

A. Room Representation

We operate on residential room-level layouts extracted from professional BIM projects, expressed in a local coordinate frame normalized for position and scale. Each room is decomposed into a *room envelope* and *room contents*: $r_{\text{env}} = (y^{\text{topo}}, y^{\text{layout}}, \mathcal{E}, \mathcal{D}, \mathcal{W})$, $r_{\text{ent}} = (\mathcal{P}, \mathcal{C})$. Here y^{topo} is a discrete room-type token, y^{layout} aggregates global scalars (area, perimeter); $\mathcal{E}, \mathcal{D}, \mathcal{W}$ are walls, doors, and windows; $\mathcal{P}, \mathcal{C}$ are props and casework. The encoder consumes only the envelope, while the decoder generates the entity sequence.

a) Walls and openings: Walls are N_E directed segments $e_j = (x_j^{(1)}, x_j^{(2)}, a_j^{\text{wall}})$ whose counter-clockwise order defines the room polygon. Doors and windows are wall-hosted: each door $d_k = (j_k, t_k, w_k, a_k^{\text{door}})$ stores the supporting edge index, a normalized position $t_k \in [0, 1]$ along that edge, opening width, and categorical attributes (family, swing). Windows

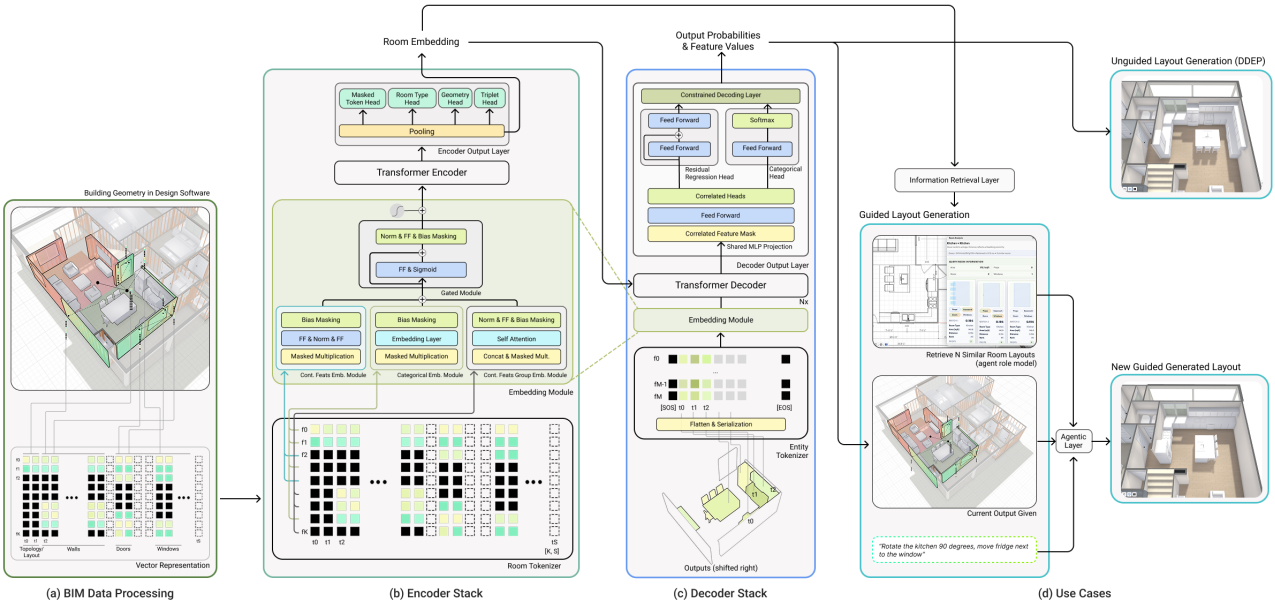

Fig. 2: Model overview. (a) BIM data extraction and assembly into a discrete set of BIM-Token Bundles. (b) The encoder stack processes the tokenized attribute–feature matrix and outputs a room representation. (c) The decoder stack consumes the room representation as memory for cross-attention and the entity sequence as input, trained with next-token prediction. (d) Downstream use cases. Retrieval (Section IV-D) and unguided generation (Sections IV-B and IV-C) are evaluated in this paper; the user-guided path through an agentic layer is a prospective integration that our BIM-native output tokens enable but that we do not evaluate here.

are defined analogously. This parameterization is invariant to absolute translation and scale, and it preserves the BIM convention that many architectural elements are hosted by, and edited relative to, a wall.

b) Entities: Each room-content entity $q \in \mathcal{P} \cup \mathcal{C}$ is parameterized as $q = (c_q, j_q, t_q, \delta_q, s_q, \rho_q, a_q^{\text{ent}})$, where c_q is the entity type, j_q is the supporting edge, t_q is the wall-relative position, δ_q is the lateral offset from the wall, s_q is size, ρ_q is rotation (props only), and a_q^{ent} collects further categorical attributes.

B. BIM Tokenization and Mixed-Type Embedding

a) BIM-Token Bundles: We map each room to two sequences of *BIM-Token Bundles*: an envelope sequence $(\tau^{\text{CLS}}, \tau^{\text{t}}, \tau^{\text{l}}, \tau_1^{\text{e}}, \ldots, \tau_1^{\text{d}}, \ldots, \tau_1^{\text{w}}, \ldots, \tau^{\text{EOS}})$ for the encoder, and an entity sequence $(\tau^{\text{SOS}}, \tau_1^{\text{ent}}, \ldots, \tau^{\text{EOS}})$ for the decoder. Each bundle corresponds to a single logical element (topology, layout, wall, door, window, or entity).

b) Attribute–feature matrix: We realize these heterogeneous sequences as a sparse attribute–feature matrix $X \in \mathbb{R}^{F \times S}$, where each column $X_{:,s}$ encodes one bundle and each row corresponds to one feature (e.g., *token_type_id*, *token_id*, edge endpoints, edge length, t-value, δ-offset, size, rotation). Only a subset of features is active for any given token type; inactive entries are filled with a sentinel padding value $p{=}{-}100$. This differs from flat field serialization: a token remains a complete BIM element, and its active rows expose the typed attributes needed to edit that element later. This sparse layout lets the same backbone handle topology tokens, wall tokens, opening tokens, and entity tokens uniformly without forcing all element types into a lossy common schema. Encoder and decoder use disjoint feature sets via X^{enc} and X^{dec}.

c) Mixed-type embedding: A shared FeatureEmbedding module turns X into dense token vectors. For each active feature f: *(i)* categorical features (type/family identifiers, edge indices) use learnable embedding tables; *(ii)* scalar continuous features (area, perimeter, length) use small MLPs; *(iii)* grouped continuous features (e.g. edge endpoint pairs, opening corner distances) use a *MultiContinuousEmbedding* that embeds each scalar in the group and aggregates via self-attention. A valid-feature mask $m_{f,s} = \mathbf{1}[X_{f,s} \neq p]$ zeros out contributions from inactive features (including the bias terms of any linear layers). The bundle embedding is the sum of all active feature embeddings, $e_s = \sum_f m_{f,s}\, E_f(X_{f,s}) \in \mathbb{R}^d$, optionally augmented with learned positional embeddings.

C. Backbone and Operating Modes

We use a standard Transformer encoder–decoder backbone with two operating modes that share weights.

a) Encoder-only mode: Given the envelope embeddings E^{enc}, the encoder produces memory $M = \text{Enc}_\theta(E^{\text{enc}}) \in \mathbb{R}^{S_{\text{enc}} \times d}$. We pool at the CLS position to obtain a room embedding $z(r_{\text{env}}) = M_0$, used for retrieval and clustering. Auxiliary heads (room-type classification, masked-token prediction) operate on M.

b) Encoder–decoder mode (DDEP): The encoder produces M as above; the decoder consumes entity embeddings E^{dec} and attends causally to its own prefix and via cross-attention to M, emitting $H = \text{Dec}_\theta(E^{\text{dec}}, M)$. Per-token heads produce mixed categorical and continuous outputs (entity type, edge attachment, t, δ, size, rotation). At inference, a constrained decoding layer filters the model's output distribution to enforce BIM validity, including egress clearance and door-swing zones. This layer is a validity guard over candidate tokens, not a post-hoc layout optimizer.

c) Training: We use a two-stage schedule: *(i)* encoder pretraining with a composite loss combining room-type classification, masked-token prediction, graded triplet contrastive learning, and geometric preservation; *(ii)* encoder–decoder fine-tuning for DDEP via teacher forcing, with cross-entropy over categorical heads and MSE over continuous heads.

IV. Experiments

We evaluate the proposed tokenization and backbone on a corpus of professional residential BIM scenes (single-family homes with typed rooms).

A. Setup, Splits, and Metrics

We use two complementary protocols whose results are not directly comparable.

Controlled same-data benchmark (lead). A frozen split of $15{,}740/2{,}009/1{,}826$ rooms (train/val/test) across five room types (`bathfull`, `bedroom`, `laundry`, `living`, `master_bed`) with shared ontology normalization, canonical furniture dimensions, and a unified evaluation harness. All methods are trained from scratch under a matched budget of 200 epochs with no method-specific tuning and evaluated on the shared 1,225-room geometry-valid intersection. This protocol is intentionally smaller than production DDEP; its purpose is to isolate architectural effects under matched data conditions rather than to report our best deployment configuration.

Frontier LLM/VLM baseline (contextual). Production DDEP follows a two-stage pipeline—encoder pre-training on the full augmented corpus (439,932 samples over 53 room types), then per-room-type encoder–decoder fine-tuning on the 75,720-sample DDEP subset (14 types)—and is compared off-the-shelf against zero-shot frontier text and vision-language models (Claude Opus/Sonnet 4.6, Claude Haiku 4.5, GPT-5.2, Gemini 3.1 Pro, Codex 5.3, Qwen 3.5) and two domain-specific systems (LayoutVLM [24], FlairGPT [26]) on a 50-room held-out set. This is a deployment-realistic floor rather than the paper's headline architectural claim; we discuss its limits in Section IV-C.

a) Metrics.: We report metrics under identical geometry and inventory specifications. The definitions below are stated over our full room ontology, since the frontier protocol spans ten room types; the controlled protocol exercises the five listed above. **Coverage** (Cov., ↑) is a semantic inventory score. For category i, predicted count p_i is compared with the allowed range $[m_i^{\min}, m_i^{\max}]$ using missing and overfill fractions $d_i = \max(0, m_i^{\min} - p_i)/\max(1, m_i^{\min})$ and $o_i = \max(0, p_i - m_i^{\max})/\max(1, m_i^{\max})$. The item score is $s_i = 1 - \text{clip}(d_i + 0.5\, o_i, 0, 1)$, and room coverage averages weighted item scores (1.0 for essential items, 0.5 for optional items) with alternative requirement groups, such as tub-or-shower in full baths or range versus oven-plus-cooktop in kitchens. A capped unsupported-extra penalty discourages hallucinated entities. Thus Cov. measures *what* was placed, not whether it is reachable or collision-free.

Navigability evaluates functional access from inward-offset door portals to essential targets on a clearance-inflated walkable grid. Beds contribute two side-access targets; sinks, toilets, ranges, and dressers contribute front-access targets. **SR** (↑) is the fraction of door–target pairs reachable by A* routing. **DF** (↓) averages the capped detour penalty $f(\rho) = \min(1, (\rho - 1)/2)$ over reachable pairs, where ρ is path length over Euclidean distance; if no pair is reachable, DF is set to 1. We report $\text{Nav} = 100(\text{SR} - 0.35\,\text{DF})$, so severely blocked rooms can score below zero.

Overlap–Clearance (OC, ↓) uses exact footprint polygons: $\text{OC} = 100(0.5\,\text{EOF} + 0.2\,\text{GOA} + 0.3\,\text{DCI})$, where EOF is per-entity overlap with eligible neighbors, GOA is total multiply occupied floor area normalized by room area, and DCI is door-clearance intrusion. Eligible overlaps include prop–prop and same-category casework collisions, while prop–casework overlaps are ignored because they often encode intentional support relations. OC must be read jointly with coverage and navigability: sparse rooms can have excellent OC while failing the program and reachability tests.

B. Controlled Same-Data Benchmark

TABLE I: Controlled same-data benchmark on the shared 1,225-room geometry-valid subset. Cov. is coverage, SR is success rate, DF is detour factor, and OC is overlap–clearance. Higher is better for Cov., Nav, and SR; lower is better for DF and OC. Values are mean ± std; bold marks the best primary method per metric.

Method	Cov. ↑	Nav ↑	SR ↑	DF ↓	OC ↓
ATISS [19] (same-data)	40.8 ± 23.6	−30.3 ± 24.0	0.03 ± 0.18	0.97 ± 0.18	**0.5 ± 2.5**
BLT [22] (same-data)	17.8 ± 20.4	−18.6 ± 42.7	0.12 ± 0.32	0.88 ± 0.31	35.7 ± 16.9
DDEP (Ours)	**54.9 ± 38.3**	**14.6 ± 56.0**	**0.36 ± 0.42**	**0.60 ± 0.44**	13.4 ± 12.5
DDEP ablations					
Wall-Only	50.8 ± 39.6	9.6 ± 54.8	0.32 ± 0.41	0.63 ± 0.44	12.7 ± 11.8
No Cont. Groups	42.6 ± 40.2	0.7 ± 52.2	0.25 ± 0.39	0.69 ± 0.43	13.4 ± 13.8
No Edge Coords	56.7 ± 39.7	14.4 ± 57.6	0.35 ± 0.43	0.60 ± 0.45	14.5 ± 12.7
Order: t first	42.6 ± 39.1	4.1 ± 54.5	0.29 ± 0.41	0.70 ± 0.41	10.6 ± 12.0
Order: edge,t desc.	46.5 ± 39.9	7.8 ± 54.8	0.31 ± 0.41	0.66 ± 0.42	13.4 ± 13.2

Table I reports the same-data benchmark. DDEP attains the strongest joint functional profile among compared methods: 54.9 coverage and 14.6 navigability, beating ATISS by +14.1 coverage and +44.9 navigability, and BLT by +37.1 coverage and +33.2 navigability. It also achieves substantially better reachability (SR 0.36 vs. 0.03/0.12) and shorter detours (DF 0.60 vs. 0.97/0.88). ATISS's near-zero OC (0.5%) reflects sparse outputs that avoid collisions while leaving almost all targets unreachable; BLT exhibits the opposite failure mode with 35.7% OC. The key result is therefore not a single scalar win but a functional tradeoff: DDEP places enough program elements to satisfy the room while keeping paths

reachable. Per-room inspection shows the strongest gains in bedrooms, living rooms, and master bedrooms; compact baths and laundries remain harder because small footprint errors quickly invalidate door clearance. ATISS targets 3D-FRONT-style scenes with axis-aligned bounding boxes and BLT targets 2D graphic-design layouts; we bridge their native outputs into our wall-referenced representation using the published descale formula plus an affine map to the physical room followed by nearest-wall projection. This bridge applies to both baselines but not to DDEP, which is native to the format, so we cannot rule out that part of the gap reflects conversion loss rather than modelling capacity. It is at least identical across ATISS and BLT, and does not touch the ablation rows. A conversion-free comparison would require retraining both baselines on wall-referenced targets.

a) Variance and model sharing.: The per-room deviations in Table I are large relative to the means because both metrics are bounded per room (Cov. in $[0, 100]$, Nav in $[-35, 100]$) with mass near the extremes: a room is typically either largely satisfied and traversable or largely blocked, and one obstructed doorway sends an otherwise sound room to the floor of the Nav scale. We therefore read Cov/Nav/OC as a joint profile rather than as individually significant scalars. The two protocols also bracket a design choice: production DDEP trains a 512-dim specialist per room type on top of a pre-trained encoder, while the controlled model is a *single* 256-dim network trained from scratch on 15,740 rooms and serving all five types. The shared, from-scratch setting is the weaker of the two, so Table I compares architectures in the harder regime. This brackets rather than measures the effect—capacity, pre-training, data, and test set all differ—so a fixed-capacity shared-versus-specialist ablation remains future work.

b) Ablations.: Five ablations isolate internal design choices within our tokenization. *No Continuous Groups* (-12.3 cov, -13.9 nav) replaces the multi-continuous embedding with independent scalar MLPs and is the most damaging single change, confirming that joint embedding of correlated geometric features is the central inductive bias. *Ordering ablations* (*t-value First*: $-12.3/-10.5$; *edge, t Desc*: $-8.4/-6.8$) show that the canonical edge-then-position ordering is also material. *Wall-Only* ($-4.1/-5.0$) confirms that opening positions help the decoder reason about clearances. *No Edge Coordinates* ($+1.8/-0.2$) is nearly neutral on coverage and navigability while OC rises slightly, indicating the model infers relative geometry from remaining features when explicit endpoints are removed. Note that all five isolate *within-tokenization* choices; the flat-serialization baseline that would test the structure itself is discussed in Section V.

C. Frontier LLM/VLM Baseline

Table II summarizes the deployment-style benchmark. Production DDEP has the only high-coverage, high-navigability, low-OC profile: it reaches 98.1 coverage, 79.2 navigability, 1.9% OC, and 3.18 ± 0.18 s latency. Frontier LLM/VLM systems close part of the semantic gap but remain less reliable spatially; domain-specific methods keep OC low only while missing large portions of the room program. To be precise about what the baselines received: every model gets the same tuned prompt with an explicit coordinate system, wall-condition guidance, a room-type-filtered entity catalog for information parity with DDEP, and a matched example of the expected output format, so the gap is not an artifact of naive prompting. It remains a calibration floor rather than an architectural ceiling because each model is run once per room with no best-of-k or chain-of-thought, and no open model is fine-tuned in-domain—the informative next comparison.

TABLE II: Deployment-style held-out benchmark on 50 production rooms. Frontier rows report the best score achieved by any evaluated text-only LLM or VLM in each metric column; domain rows are individual systems. LayoutVLM requires a furniture list, so Coverage is not applicable. Bold marks the best result per metric.

Method	Cov. ↑	Nav. ↑	OC ↓	Lat. (s) ↓
DDEP (ours)	**98.1**	**79.2**	**1.9**	**3.18**
Text LLM envelope	68.3	18.9	9.8	7–81
VLM envelope	65.6	55.5	8.2	7–102
LayoutVLM [24]	–	40.3	3.8	145
FlairGPT [26]	46.6	28.2	3.9	1134

Fig. 3: 2D diagnostic grid for representative room types. Red marks blocking or wrong placement; blue marks undesired rotation. DDEP avoids most highlighted failures while satisfying the room program.

D. Embedding Evaluation

We also evaluate SBM's encoder-only pathway against E5-Large-v2 [29], BGE-Large-v1.5 [30], and GTE-Large-v1.5 [31] using serialized room text for the baselines. Text encoders lead on within-type nDCG (E5 86.7 vs. SBM 68.2), but SBM produces substantially more coherent type-level clusters (NMI 0.726 vs. 0.371; ARI 0.437 vs. ≤ 0), which is the property used by retrieval-augmented BIM generation.

E. Qualitative Comparisons and Failure Modes

Figure 1 shows generated layouts across room types, and Fig. 3 gives a 2D diagnostic view of the same comparison. VLM baselines tend to overfill rooms with extra casework and

furniture, blocking circulation and door-swing zones. Domain-specific methods place more appropriate furniture types but still leave narrow or blocked paths around key elements. DDEP layouts satisfy the room program while maintaining clear, door-connected circulation bands and respecting wall-referenced anchoring. Characteristic DDEP failures include under-populating ambiguous open-plan rooms, dropping an entity when door clearance is over-tight, and misaligning casework groups on short wall segments.

V. Conclusion

We presented a BIM-native tokenization for room-level layout synthesis: sparse attribute–feature matrix columns for logical elements, wall-referenced coordinates, and a mixed-type embedding module for categorical, scalar, and grouped-continuous attributes. Reusing one Transformer for retrieval and DDEP, the method improves the controlled same-data benchmark against ATISS/BLT and produces stronger type-level clusters than text encoders. The core takeaway is representational: modest domain-specific sequence models over BIM-editable tokens can be an effective primitive for constraint-aware indoor layout synthesis, complementary to general-purpose LLMs/VLMs.

Several limitations bound these claims and set the agenda for future work. Our evaluation is limited to residential rooms from a proprietary corpus, which constrains external reproducibility; any future release of data or code will be announced on the project page. The LLM/VLM benchmark compares trained DDEP against zero-shot systems, so in-domain fine-tuning of an open model may close part of the gap. The ablations test *within-tokenization* choices, which makes a flat-serialization baseline that removes the attribute–feature structure entirely the next critical experiment. We also omit direct HouseDiffusion [13]/LayoutGPT [23] comparisons because their polygonal or object–relation outputs need a non-trivial bridge to wall-hosted BIM entities. Non-residential typologies, building-scale layouts, vertical constraints, and local code variation remain open, as does disentangling model capacity from data scale in the shared-versus-specialist comparison.

References

[1]

[2] C. N. Eastman, *Spatial synthesis in computer-aided building design*. Elsevier Science Inc., 1975.

[3] J. Monedero, "Parametric design: a review and some experiences," *Automation in Construction*, vol. 9, no. 4, pp. 369–377, 2000.

[4] P. Kán and H. Kaufmann, "Automated interior design using a genetic algorithm," in *Proceedings of the 23rd ACM Symposium on Virtual Reality Software and Technology*, VRST '17, (New York, NY, USA), pp. 1–10, Association for Computing Machinery, Nov. 2017.

[5] F. Flager and J. R. Haymaker, "A comparison of multidisciplinary design, analysis and optimization processes in the building construction and aerospace industries," in *A comparison of multidisciplinary design, analysis and optimization processes in the building construction and aerospace industries*, 2007.

[6] P. Merrell, E. Schkufza, Z. Li, M. Agrawala, and V. Koltun, "Interactive furniture layout using interior design guidelines," *ACM Trans. Graph.*, vol. 30, pp. 87:1–87:10, July 2011.

[7] P. Song, Y. Zheng, J. Jia, and Y. Gao, "Web3D-based automatic furniture layout system using recursive case-based reasoning and floor field," *Multimedia Tools and Applications*, vol. 78, pp. 5051–5079, Feb. 2019.

[8] K. Xu, L. Wu, Z. Wang, Y. Feng, M. Witbrock, and V. Sheinin, "Graph2seq: Graph to sequence learning with attention-based neural networks," 2018.

[9] R. Hu, Z. Huang, Y. Tang, O. van Kaick, H. Zhang, and H. Huang, "Graph2plan: Learning floorplan generation from layout graphs," *arXiv preprint arXiv:2004.13204*, 2020.

[10] H. Tanasra, T. Rott Shaham, T. Michaeli, G. Austern, and S. Barath, "Automation in Interior Space Planning: Utilizing Conditional Generative Adversarial Network Models to Create Furniture Layouts," *Buildings*, vol. 13, p. 1793, July 2023. Publisher: Multidisciplinary Digital Publishing Institute.

[11] Y. Liu and G. Wang, "Exploration of the Indoor Layout Optimization Model in Computer-Aided Visual Analysis," *Computer-Aided Design and Applications*, pp. 167–180, Aug. 2024.

[12] N. Nauata, W.-C. M. Chang, Y. Furukawa, and et al., "House-gan++: Generative adversarial layout refinement network towards intelligent computational agent," in *CVPR*, 2021.

[13] M. A. Shabani, S. Hosseini, and Y. Furukawa, "Housediffusion: Vector floorplan generation via a diffusion model with discrete and continuous denoising," 2022.

[14] S. Hu *et al.*, "MiDiffusion: Mixed diffusion for 3d indoor scene synthesis," *arXiv preprint arXiv:2405.21066*, 2024.

[15] X. Sun *et al.*, "SemLayoutDiff: Semantic layout generation with diffusion models," *arXiv preprint arXiv:2508.18597*, 2025.

[16] H. T. Nguyen, Y. Chen, V. Voleti, V. Jampani, and H. Jiang, "Housecrafter: Lifting floorplans to 3d scenes with 2d diffusion model," in *arXiv preprint arXiv:2406.20077*, 2024.

[17] D. Srivastava *et al.*, "Lay-your-scene: Natural scene layout generation with diffusion transformers," *arXiv preprint arXiv:2505.04718*, 2025.

[18] X. Ran *et al.*, "Directlayout: Direct numerical layout generation for 3d indoor scene synthesis," *arXiv preprint arXiv:2506.05341*, 2025.

[19] D. Paschalidou, A. Kar, M. Shugrina, K. Kreis, A. Geiger, and S. Fidler, "Atiss: Autoregressive transformers for indoor scene synthesis," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[20] X. Wang, C. Yeshwanth, and M. Nießner, "Sceneformer: Indoor scene generation with transformers," *arXiv preprint arXiv:2012.09793*, 2020.

[21] J. Liu, W. Xiong, I. Jones, Y. Nie, A. Gupta, and B. Ouguz, "Cliplayout: Style-consistent indoor scene synthesis with semantic furniture embedding," *ArXiv*, vol. abs/2303.03565, 2023.

[22] X. Kong, L. Jiang, H. Chang, H. Zhang, Y. Hao, H. Gong, and I. Essa, "Blt: Bidirectional layout transformer for controllable layout generation," in *Computer Vision – ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part XVII*, (Berlin, Heidelberg), p. 474–490, Springer-Verlag, 2022.

[23] W. Feng, W. Zhu, T.-J. Fu, V. Jampani, A. R. Akula, X. He, S. Basu, X. E. Wang, and W. Y. Wang, "Layoutgpt: Compositional visual planning and generation with large language models," in *Advances in Neural Information Processing Systems*, 2023.

[24] F.-Y. Sun, W. Liu, S. Gu, D. Lim, G. Bhat, F. Tombari, M. Li, N. Haber, and J. Wu, "Layoutvlm: Differentiable optimization of 3d layout via vision-language models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 29469–29478, 2025.

[25] Z. Zong, G. Chen, Z. Zhan, F. Yu, and G. Tan, "Housetune: Two-stage floorplan generation with LLM assistance," 2024.

[26] G. Littlefair, N. S. Dutt, and N. J. Mitra, "Flairgpt: Repurposing llms for interior designs," 2025. EUROGRAPHICS 2025.

[27] X. Li, Y. Sun, and Z. Sha, "Llm4cad: Multi-Modal large language models for three-dimensional computer-aided design generation," in *Proceedings of the ASME 2024 International Design Engineering Technical Conferences and Computers and Information in Engineering Conference (IDETC/CIE 2024)*, vol. 88407, p. V006T06A015, ASME, 2024.

[28] J. Yin, P. Zeng, J. Zhong, P. Li, M. Zhang, R. Luo, and S. Lu, "Floorplan-deepseek (fpds): A multimodal approach to floorplan generation using vector-based next room prediction," *arXiv preprint*, vol. arXiv:2506.21562, 2025.

[29] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, and F. Wei, "Text embeddings by weakly-supervised contrastive pre-training," *arXiv preprint arXiv:2212.03533*, 2022.

[30] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C-pack: Packaged resources to advance general chinese embedding," 2023.

[31] Z. Li, X. Zhang, Y. Zhang, D. Long, P. Xie, and M. Zhang, "Towards general text embeddings with multi-stage contrastive learning," *arXiv preprint arXiv:2308.03281*, 2023.